브랜딩 트렌드 AI 비서 개발 결과 보고서

AI 응용 미션 2 나만의 AI 비서 구축

작성자 조민경 최종 수정일 2026 년 10 월 21 일

미션 1 에서 분석한 브랜딩 자료를 사용자가 직접 조회하고 AI 에게 질문할 수 있는 웹서비스로 만들었습니다. 이 보고서는 개발 목적에서 시작해 데이터 준비와 기능 구현, 실제 검증과 문제 해결, 배운 점으로 이어집니다.

1 개발 목적과 완성 결과

미션 1 에서는 퍼스널브랜딩·이미지메이킹·스피치의 검색 관심도 변화를 분석했습니다. 이번에는 엑셀을 직접 열어 계산하지 않고도 사용자가 기간별 그래프를 보고 궁금한 점을 질문할 수 있도록 만들고자 했습니다. 분석 결과를 한 번 읽는 데서 더 나아가, 필요한 자료를 직접 찾아 활용하는 서비스로 연결했습니다.

완성한 서비스에서는 데이터 요약과 그래프를 확인하고, 자료를 추가·조회·수정·삭제하며, AI 와 나눈 대화를 저장하고 다시 불러올 수 있습니다. AI 가 필요한 기간의 자료를 조회하는 기능과 외부 AI 프로그램의 데이터 조회 연결도 확인했습니다.

제가 맡은 역할

저는 필요한 기능과 확인할 내용을 정하고, Codex 의 도움으로 코드를 구현하고 수정했습니다. 실행 결과를 확인하면서 문제가 있는 부분을 다시 점검했습니다. 직접 모든 코드를 작성한 것은 아니며, AI 가 만든 결과가 요구사항에 맞게 작동하는지 확인하는 과정을 함께 수행했습니다.

결과를 확인할 주소

서비스 화면 https://branding-trend-ai-assistant-fronten.vercel.app
백엔드 API https://branding-trend-ai-assistant.onrender.com
API 설명서 https://branding-trend-ai-assistant.onrender.com/docs
GitHub https://github.com/coolgirl2046/branding-trend-ai-assistant

확인 결과 데이터 558 건, 자동화 테스트 108 개 통과, 최종 코드 배포 후 실제 AI 답변을 확인했습니다.

1.1 이 보고서에 나오는 용어

이 보고서에는 개발 용어가 자주 나옵니다. 개발을 전공하지 않은 분도 읽을 수 있도록 자주 쓰는 용어를 먼저 쉬운 말로 정리했습니다.

용어	쉬운 설명
프런트엔드	사용자가 눈으로 보고 누르는 화면 부분입니다. 버튼, 그래프, 채팅창이 여기에 해당합니다.
백엔드	화면 뒤에서 요청을 받아 자료를 찾고 저장하고 AI 에 질문하는 프로그램입니다. 식당으로 치면 손님에게 보이지 않는 주방입니다.
API	프로그램끼리 요청과 결과를 주고받는 창구입니다. 식당의 주문 창구처럼 정해진 방식으로 요청하면 결과를 돌려줍니다.
데이터베이스(DB)·Firestore	자료를 보관하는 저장소입니다. 이 프로젝트는 구글의 온라인 저장소인 Firestore 를 사용했습니다.
컬렉션·필드	컬렉션은 비슷한 자료를 모아 두는 폴더, 필드는 자료 한 건에 들어가는 항목 이름(날짜, 키워드 등)입니다.
CRUD	자료를 추가(Create)·조회(Read)·수정(Update)·삭제(Delete)하는 네 가지 기본 기능입니다.
JSON	이름과 값을 짝지어 적는 자료 형식입니다. "count": 186 은 '개수가 186'이라는 뜻입니다.
배포	내 컴퓨터에서 만든 프로그램을 인터넷에 올려 누구나 주소로 접속할 수 있게 하는 일입니다. 화면은 Vercel, 백엔드는 Render 에 배포했습니다.
Ready·Live·Production	배포 화면에 나오는 상태 표시입니다. Ready 와 Live 는 배포 성공, Production 은 실제 서비스 주소에 반영된 배포라는 뜻입니다.
커밋·GitHub	커밋은 코드 변경 내용을 저장한 기록 한 건으로, 7bc1366 같은 번호로 구분합니다. GitHub 는 이 기록을 온라인에 보관하는 곳입니다.
환경변수	비밀키 같은 설정값을 코드 밖 별도 공간에 두고, 프로그램이 실행될 때 읽어 오게 하는 방식입니다.
시스템 프롬프트	AI 에게 질문보다 먼저 주는 기본 안내문입니다. 답변 규칙과 참고 자료가 들어갑니다.
컨텍스트 주입	질문할 때 제 자료의 요약을 시스템 프롬프트에 함께 넣어 주는 방식입니다. AI 에게 참고 메모를 건네주는 것과 같습니다.

용어	쉬운 설명
토큰	AI 가 글을 나눠 처리하는 단위입니다. 글자 조각이라고 생각하면 되며, 답변 길이와 이용료의 기준이 됩니다.
Function Calling	AI 가 자료를 직접 찾지 않고, 정해진 조회 기능을 써 달라고 프로그램에 부탁하는 기능입니다.
MCP	외부 AI 프로그램이 같은 조회 기능을 연결해 쓸 수 있게 하는 공통 연결 규칙입니다. 여러 기기를 하나의 충전 단자로 연결하는 것과 비슷합니다.
CORS	브라우저가 다른 주소의 서버에서 온 응답을 읽어도 되는지 확인하는 규칙입니다. 허용한 주소의 화면만 백엔드 응답을 읽을 수 있습니다.
Pydantic	들어온 자료가 정해진 형식에 맞는지 자동으로 검사하는 도구입니다. 날짜 형식이나 숫자 범위가 틀리면 저장하지 않습니다.
HTTP 상태 코드	요청 결과를 알려 주는 번호입니다. 404 는 '찾을 수 없음', 422 는 '입력 형식 오류', 500 번대는 '서버 쪽 문제'를 뜻합니다.
콜드스타트	한동안 쉬고 있던 서버가 첫 요청을 받고 깨어나느라 응답이 늦어지는 현상입니다.
Swagger	백엔드에 보낼 수 있는 요청 목록을 보여주고 직접 시험해 볼 수 있는 설명서 화면입니다.
자동화 테스트·모의 응답	자동화 테스트는 프로그램이 정해진 대로 동작하는지 자동으로 검사하는 코드입니다. 외부 AI 나 DB 대신 테스트용 가짜 응답(모의 응답)을 쓰기도 합니다.
프롬프트 인젝션	AI 가 참고 자료 속에 숨겨진 엉뚱한 지시에 속아 따르는 문제입니다.
상대 검색 관심도	검색 횟수가 아니라, 같은 조건에서 검색 관심의 크기를 0~100 사이로 비교한 지수입니다.
이동평균	연속된 값 몇 개의 평균을 한 칸씩 옮겨 가며 구해, 들쭉날쭉한 흐름을 부드럽게 보는 방법입니다.

위 용어는 보고서 본문에서도 처음 나올 때 괄호나 짧은 문장으로 다시 풀어 썼습니다.

2 사용한 데이터와 숫자의 의미

사용한 자료는 네이버 데이터랩의 주간 상대 검색 관심도입니다. 서비스에 표시된 기간은 2023 년 1 월 2 일부터 2026 년 7 월 20 일까지이며, 세 키워드를 186 주 동안 기록한 총 558 건입니다. 미션의 데이터 100 건 이상 조건을 충족합니다.

엑셀 자료를 서비스에서 사용하는 과정

엑셀 자료를 날짜·키워드·관심도 값으로 정리하고 Firestore 라는 온라인 데이터 저장소에 넣어 사용하도록 구성했습니다. 한 주의 퍼스널브랜딩 값 한 개가 데이터 한 건입니다. 세 키워드가 각각 기록되므로 186 주 × 3 개 키워드 = 558 건이 됩니다.

저장소의 변환 코드와 테스트에서 이 구조를 확인했고, 운영 화면에서도 실제 558 건을 확인했습니다.

상대 검색 관심도는 검색 횟수가 아닙니다

관심도 100 은 100 번 검색했다는 뜻이 아닙니다. 같은 조회 조건에서 검색 관심도의 크기를 상대적으로 나타낸 지수입니다. 따라서 값이 낮아졌다고 실제 검색 건수가 정확히 그만큼 줄었다고 설명할 수는 없습니다.

요약 평균과 그래프 이동평균의 차이

구분	계산과 목적
대시보드 요약	키워드별 최신 관측값 4 개의 평균입니다. 연속된 주간 자료이면 최근 4 주 평균에 해당합니다. 최신 상태를 요약합니다.
그래프의 4 주 이동평균	연속된 주간 값 4 개씩을 묶어 평균을 내고, 묶음을 한 주씩 옮깁니다. 주별 흔들림을 줄여 흐름을 보기 위한 선입니다.

계산 예시로 주간 값이 10, 20, 30, 40, 50 이라면 처음 네 값의 평균은 25 이고, 다음 네 값의 평균은 35 입니다. 이 예시는 계산법을 보여주는 가상의 숫자이며 실제 브랜딩 자료는 아닙니다.

여기서 '최근'은 오늘 날짜가 아니라 저장된 자료의 마지막 관측 시점을 기준으로 합니다. 이 서비스가 현재 인터넷 검색 트렌드를 실시간으로 수집하는 것은 아닙니다.

2.1 요약 정보와 추세 판단

/api/data/summary 는 저장된 자료를 한눈에 볼 수 있게 정리해 돌려주는 요청 주소입니다. 전체 기간과 자료 수, 키워드별 개수·전체 평균·최솟값(가장 작은 값)·최댓값(가장 큰 값)·최신 값·최근 4 개 평균·이전 4 개 평균·평균 차이·추세·순위를 돌려줍니다. 기본 평균 외에 최솟값·최댓값, 최근과 이전 평균의 차이, 순위를 추가 통계로 넣었습니다.

추세의 기준

최근 4 개 평균에서 직전 4 개 평균을 뺀 차이가 0.5 포인트보다 크면 상승, -0.5 포인트보다 작으면 하락, 그 사이(절댓값 0.5 포인트 이하)면 유지입니다. 쉽게 말해 0.5 점보다 많이 오르면 상승, 많이 내리면 하락, 그보다 적게 움직이면 유지로 봅니다. 0.5 는 이 프로그램에서 정한 기준이며 통계적으로 의미 있는 차이라는 뜻은 아닙니다.

응답 구조의 실제 계산 예시

다음은 원본 엑셀 558 건을 저장소의 요약 코드로 계산한 결과 중 일부입니다. 운영 중인 서비스의 응답을 캡처한 것이 아니라 같은 코드로 다시 계산한 발췌입니다. JSON 은 이름과 값을 짝지어 적는 자료 형식으로, 예를 들어 "total_count": 558 은 '전체 자료 수가 558 건'이라는 뜻입니다.


```
{
  "start_date": "2023-01-02",
  "end_date": "2026-07-20",
  "total_count": 558,
  "trend_rule": {
    "stability_threshold": 0.5
  },
  "keywords": {
    "이미지메이킹": {
      "count": 186,
      "recent_4week_average": 4.3541,
      "previous_4week_average": 4.71237,
      "difference": -0.35828,
      "trend": "유지"
    }
  }
```

}

이미지메이킹은 4.71237 에서 4.35410 으로 약 0.35828 포인트 감소했지만, 정한 허용 폭 안에 있으므로 추세는 유지입니다. 수치 감소와 추세 분류는 서로 다른 설명입니다.

3 서비스 구성과 작동 과정

사용자가 보는 화면, 화면 뒤에서 일을 처리하는 프로그램, 자료를 보관하는 저장소가 역할을 나누어 작동합니다. AI 는 저장된 자료의 요약을 전달받고, 필요한 경우 정해진 조회 기능으로 자료를 더 가져옵니다.

기술 이름	제 프로젝트에서 하는 일
HTML CSS JavaScript 프런트엔드	사용자가 보는 화면입니다. 버튼·그래프·데이터 목록·채팅창을 만들었습니다.
Python FastAPI 백엔드	화면 뒤에서 일하는 프로그램입니다. 화면의 요청을 받아 자료를 조회하거나 저장하고 AI 에 답변을 요청합니다.
Firestore 데이터베이스	검색 관심도 자료와 대화 기록을 보관하는 구글의 온라인 저장소(데이터베이스)입니다.
GPT API	API 는 프로그램끼리 요청과 결과를 주고받는 창구입니다. GPT API 를 통해 프로그램에서 AI 에 질문을 보내고 답을 받아옵니다.
Render 와 Vercel	만든 프로그램을 인터넷에 올려 누구나 주소로 접속할 수 있게 해 주는 서비스입니다(배포). Render 에는 백엔드를, Vercel 에는 화면을 올렸습니다.
Swagger	백엔드에 보낼 수 있는 요청 목록을 보여주고 직접 시험해 볼 수 있는 설명서 화면입니다.

개발 환경과 로컬 실행

Python 3.10 이상에서 가상환경(venv)을 만들어 구성했습니다. 가상환경은 이 프로젝트에 필요한 프로그램 부품(패키지)만 따로 담아 두는 작업 공간입니다. 필수 패키지는 fastapi, uvicorn, firebase-admin, openai, python-dotenv 이며, 입력값 검사와 추가 기능을 위해 Pydantic, openpyxl, MCP SDK 도 사용합니다. 화면은 HTML·CSS·JavaScript 만으로 만들었고, 미리 만들어진 화면 개발 틀(프레임워크)은 사용하지 않았습니다. 아래 명령은 Windows PowerShell(명령 입력 창)에서 프로젝트 폴더를 기준으로 입력합니다.

백엔드 실행

```
python -m venv .venv
.venv₩Scripts₩Activate.ps1
python -m pip install -r backend/requirements.txt
python -m uvicorn backend.app.main:app --reload --host 127.0.0.1 --port 8000
```

프런트엔드 실행

python -m http.server 5500 --directory frontend

브라우저 주소 http://localhost:5500

질문 한 번이 처리되는 순서

① Firestore 에서 현재 데이터 요약을 계산합니다.

② 요약 JSON 을 시스템 프롬프트(AI 에게 먼저 주는 답변 규칙과 참고 자료)에 넣고, 기존 대화와 새 질문을 구성합니다.

③ GPT API 를 호출합니다. 원자료가 필요하면 읽기 전용 도구를 추가로 실행합니다.

④ 응답 성공 후 질문과 답변을 conversations 문서의 messages 에 자동 저장합니다.

자료를 바꾸는 기능과 AI 가 조회하는 기능

사용자는 데이터 관리 화면에서 자료를 추가·조회·수정·삭제할 수 있습니다. 이를 CRUD 라고 부릅니다. 반면 AI 에 연결한 도구는 읽기 전용입니다. AI 가 이 조회 도구로 원본 자료를 수정하거나 삭제하도록 구성한 것은 아닙니다.

3.1 요청 접수와 처리 로직의 분리

라우터는 어떤 주소로 어떤 요청이 왔는지 확인하고 필요한 처리 함수를 연결하는 접수 창구입니다. 서비스는 요약 계산이나 AI 요청처럼 실제 일을 수행합니다. 주소별 접수와 재사용할 처리 로직을 나누면 한 기능을 수정할 때 영향을 확인하기 쉽고, 계산 부분만 별도로 테스트할 수 있습니다.

실제 폴더와 역할

backend/app/main.py 앱 시작과 CORS 및 라우터 연결

backend/app/routers/data.py 자료 CRUD 와 요약 요청 접수

backend/app/routers/conversations.py 대화 목록과 상세 조회 및 삭제

backend/app/routers/chat.py 질문 처리와 대화 자동 저장

backend/app/services/summary.py 통계와 추세 계산

backend/app/services/data_summary.py DB 자료를 계산 함수에 전달

backend/app/services/ai.py 시스템 프롬프트와 GPT 호출

backend/app/services/trend_tools.py 읽기 전용 자료 조회

backend/app/schemas.py 입력과 응답 형식 정의

Firestore 컬렉션 구조

컬렉션	주요 필드	역할
data	date, keyword, value, memo, created_at, updated_at	주간 상대 검색 관심도 자료 저장
conversations	title, messages(role, content, created_at), created_at, updated_at	대화 제목과 사용자·AI 메시지 저장

컬렉션은 비슷한 자료를 모아 두는 폴더와 같고, 필드는 자료 한 건에 들어가는 항목 이름입니다. 과제 예시의 date·value·memo 에 keyword 를 추가한 이유는 같은 날짜에 퍼스널브랜딩·이미지메이킹·스피치 세 자료를 각각 구분해 저장하기 위해서입니다. created_at 과 updated_at 은 자료를 만든 시각과 고친 시각을 자동으로 기록하는 항목입니다.

Pydantic 으로 잘못된 입력 차단

Pydantic 은 들어온 자료가 정해진 형식에 맞는지 자동으로 검사해 주는 도구입니다. DB 에 저장하기 전에 Pydantic 으로 입력을 검사합니다. 날짜는 실제 존재하는 YYYY-MM-DD 형식, 키워드는 세 가지 중 하나, 관심도는 0~100 사이의 숫자만 허용합니다. 메모는 1,000 자, 질문은 10,000 자로 제한하고 정해 두지 않은 항목이 들어오면 거부합니다.

예를 들어 관심도에 -1 이나 문자 "25"를 보내면 저장하지 않고 '입력 형식이 잘못됐다'는 뜻의 HTTP 422 오류로 응답합니다. 이 프로젝트는 숫자처럼 보이는 문자도 받지 않고 실제 숫자만 받습니다. 오류 안내에는 사용자가 보낸 값이나 프로그램 내부 정보를 그대로 보여주지 않도록 main.py 에서 처리합니다.

코드 근거 main.py 의 include_router 와 RequestValidationError 처리, schemas.py 의 TrendDataFields 및 validate_finite_value, services 의 순수 계산 함수

3.2 배포 연결과 오류 처리

CORS 가 필요한 이유

CORS 는 브라우저가 다른 주소의 서버에서 온 응답을 읽어도 되는지 확인하는 보안 규칙입니다. Vercel 화면과 Render 백엔드는 인터넷 주소가 다르기 때문에, 허용할 주소 목록을 미리 정해

두어야 화면이 백엔드의 응답을 읽을 수 있습니다. 실제 변수명은 CORS_ORIGINS 입니다. 쉼표로 구분한 주소를 main.py 에서 읽으며, 값이 없으면 내 컴퓨터에서 개발할 때 쓰는 주소만 허용합니다. 기본 목록에는 http://localhost:5500 과 http://127.0.0.1:5500 이 포함되어 있어 위의 로컬 실행 명령으로 접속해도 차단되지 않습니다. 다만 CORS 는 브라우저의 주소 확인 규칙일 뿐, 로그인처럼 사용자를 확인하는 기능은 아닙니다.

비밀키와 공개 API 주소 구분

환경변수는 비밀키 같은 설정값을 코드 밖의 별도 공간에 두고, 프로그램이 실행될 때 읽어 오게 하는 방식입니다. AI 키와 Firebase 인증 정보는 백엔드의 환경변수로 관리합니다. 비밀값을 적어 두는 .env 파일과 인증 파일은 GitHub 에 올라가지 않도록 제외했고, .env.example 에는 실제 비밀값 없이 설정 항목 이름만 남겼습니다. 화면(Vercel)은 배포할 때 백엔드 주소인 API_BASE_URL 을 읽어 runtime-config.js 파일을 만들고, 이 주소로 요청을 보냅니다. 이 주소는 비밀이 아니라 누구나 볼 수 있는 값입니다. 내 컴퓨터에서 실행할 때는 127.0.0.1:8000(내 컴퓨터 주소)을 사용합니다. 10 월 21 일 Production 환경변수 설정과 재배포 성공을 확인했습니다.

콜드스타트와 사용자 안내

Render 서버는 한동안 요청이 없으면 쉬는 상태가 됩니다. 이후 첫 요청이 오면 서버가 깨어나는 데 시간이 걸려 응답이 늦어지는데, 이를 콜드스타트라고 합니다. 그래서 첫 데이터 로딩 중 "첫 접속은 서버 준비로 지연될 수 있습니다. 잠시 기다려 주세요."라는 안내가 표시되도록 보완했습니다. 일반 로딩 및 연결 오류 안내와 함께 사용자가 기다리는 이유를 이해하도록 했습니다.

호출 제한과 예외 처리

AI 호출이 실패했을 때 자동으로 여러 번 다시 시도하지 않도록 했고(max_retries=0), AI 가 한 번의 질문에서 조회 도구를 최대 3 번까지만 쓰도록 제한했습니다(MAX_TOOL_CALLS=3). 또 AI 답변 길이를 호출당 기본 800 토큰으로 제한했습니다(AI_MAX_OUTPUT_TOKENS). 토큰은 AI 가 글을 나눠 처리하는 단위로, 글자 조각이라고 생각하면 됩니다. 이 제한은 max_completion_tokens 설정으로 전달하며, 호환 API 에서는 max_tokens 를 선택할 수 있습니다. 도구 사용 뒤 이어지는 요청에도 같은 제한이 전달되는지 테스트했습니다. 다만 AI 에 보내는 글의 길이나 전체 이용료까지 제한하는 것은 아닙니다.

오류는 원인에 따라 번호(HTTP 상태 코드)를 나눠 알려 줍니다. 설정 문제는 503, AI 응답 실패는 502, 없는 대화는 404, 내부 처리 실패는 500 입니다. 오류의 자세한 내부 내용이나 비밀정보는 사용자에게 보여주지 않고 이해하기 쉬운 안내 문구만 돌려줍니다.

코드 근거 main.py, services/ai.py, routers/chat.py, frontend/js/config.js 및 api.js

4 AI 의 데이터 활용 방식

자료를 전달하는 방식

일반적인 AI 채팅창에 질문만 보내면 AI 는 제 저장소에 무엇이 있는지 알 수 없습니다. 그래서 질문을 보낼 때 현재 자료의 요약도 함께 전달했습니다. 이를 컨텍스트 주입이라고 부르며, 쉽게 말하면 답변에 참고할 메모를 질문과 함께 건네주는 방식입니다.

AI 에는 숫자를 임의로 만들지 않고, 자료에서 확인되는 사실과 해석·제안을 구분하도록 안내했습니다. 자료만으로 이유를 알 수 없으면 원인을 단정하지 않고, 필요한 경우 데이터 기준일을 밝히도록 했습니다.

Function Calling 은 필요한 자료를 가져오게 하는 기능입니다

Function Calling 은 AI 가 자료를 직접 찾는 대신 '이 조건으로 자료를 찾아 달라'고 프로그램에 부탁하는 기능입니다. 예를 들어 특정 키워드의 특정 기간 자료가 필요하면, AI 가 정해진 조회 기능을 사용해 달라고 프로그램에 요청합니다. 프로그램은 허용된 키워드와 날짜인지 확인한 뒤 Firestore 에서 자료를 가져옵니다. AI 는 그 결과를 받아 답변에 활용합니다.

get_weekly_interest_data 도구가 실제 환경에서 한 번 호출되어 주간 자료 5 건을 반환한 기록을 확인했습니다.

MCP 는 외부 AI 프로그램에도 조회 기능을 연결하는 규격입니다

웹서비스 안의 채팅 외에도 다른 AI 프로그램이 같은 자료를 조회할 수 있도록 MCP 서버를 구성했습니다. MCP 는 AI 프로그램과 도구를 연결할 때 사용하는 공통 규격입니다. 제 프로젝트에서는 같은 읽기 전용 데이터 조회 기능을 재사용했습니다.

외부 AI 프로그램인 Claude Code 에서 서버 연결과 query_weekly_interest 도구를 확인했습니다. 퍼스널브랜딩의 2026 년 1 월 1 일부터 3 월 31 일까지를 한 번 조회해 Firestore 자료 13 건을 받았고 오류는 없었습니다.

두 기능을 제 말로 구분하면

"Function Calling 은 서비스 안에서 AI 가 필요한 자료를 조회해 달라고 요청하는 기능입니다. MCP 는 외부 AI 프로그램에서도 그 조회 기능을 연결해서 쓸 수 있게 하는 규격입니다."

4.1 시스템 프롬프트와 자동 저장

컨텍스트 주입은 제 데이터를 참고 자료로 전달하는 방식입니다. DB 요약 조회 → 시스템 프롬프트에 삽입 → GPT 호출 → 응답 성공 후 conversations 에 자동 저장 순서로 진행됩니다. 시스템 프롬프트는 AI 에게 질문보다 먼저 주는 기본 안내문입니다. 사용자가 별도의 저장 버튼을 누르지 않아도 질문과 답변이 messages 에 쌓입니다.

실제 시스템 프롬프트 발췌

"당신은 브랜딩 트렌드 데이터에 근거해 답하는 AI 비서입니다."
"- 이 데이터는 네이버 데이터랩의 상대 검색 관심도 지수이며 절대 검색량이 아니다."
"- 제공된 숫자를 임의로 만들거나 변경하지 않는다."
"- 데이터에서 직접 확인되는 내용은 관찰/사실로 구분한다."
"- 전략적 해석이나 추천은 해석/제안으로 구분한다."
"아래 JSON 은 현재 Firestore 데이터에서 계산한 요약이다."

코드에서는 build_system_prompt 라는 함수(정해진 일을 하는 코드 묶음)가 위 안내문 뒤에 요약 JSON 을 붙여 시스템 프롬프트를 만듭니다. 이어서 generate_answer 함수가 이 내용을 'system' 역할, 즉 AI 가 먼저 따르는 기본 안내로 넣고 기존 대화와 새 질문을 함께 AI 에 보냅니다. AI 를 새로 학습시키는 것이 아니라, 질문할 때마다 참고 자료와 답변 규칙을 함께 건네주는 구조입니다.

저장되는 내용

새 대화이면 conversations 문서를 만들고 title·messages·created_at·updated_at 을 저장합니다. 기존 대화이면 해당 문서의 messages 와 updated_at 을 갱신합니다. messages 에는 사용자 질문과 AI 답변, 각각의 시각이 들어갑니다. AI 호출이 실패하면 성공한 답변처럼 저장하지 않고 오류로 응답합니다.

코드 근거 routers/chat.py 의 calculate_firestore_summary → build_system_prompt → generate_answer → document.set 또는 document.update, services/ai.py 의 system 역할 메시지

4.2 컨텍스트 주입의 장점과 단점

요약만 전달하는 이유

컨텍스트 주입은 4.1 에서 설명한 것처럼 질문할 때 제 자료의 요약을 시스템 프롬프트에 함께 전달하는 방식입니다. 558 건의 원자료를 모두 보내지 않고 기간·개수·평균·추세처럼 답변에 필요한 통계만 정리해 전달합니다. 아래는 이 방식을 사용하며 확인한 장점과 주의할 점입니다.

장점

제 자료에 맞춘 답변이 가능합니다. 일반적인 AI 는 제 저장소에 어떤 자료가 있는지 알 수 없습니다. 요약을 함께 전달하면 "스피치의 최근 4 개 관측값 평균은 37.79 이며 이전보다 하락했다"처럼 실제 자료를 기준으로 답할 수 있습니다.

답변의 근거를 확인하기 쉽습니다. 화면의 데이터 요약과 AI 가 참고하는 요약은 같은 계산 결과입니다. 따라서 화면에 표시된 평균과 AI 가 말한 숫자를 비교해 검증할 수 있습니다. 다만 같은 자료를 참고해도 표현과 해석은 답변마다 달라질 수 있습니다.

단점과 대응

요약에 없는 세부 질문에는 답하기 어렵습니다. 요약에는 주별 값이 모두 들어 있지 않으므로, 예를 들어 특정 주의 관심도를 묻는 질문은 요약만으로 답할 수 없습니다. 이를 보완하기 위해 필요한 기간의 원자료를 읽기 전용 도구로 조회하는 Function Calling 을 추가했습니다.

전달하는 내용이 길수록 시간과 비용이 늘어납니다. AI 에 보내는 글이 길어지면 응답 시간과 이용료가 늘 수 있어 필요한 통계만 요약해 전달합니다. 답변 길이도 호출당 800 토큰으로 제한했지만, 이것이 입력을 포함한 전체 이용료를 고정하지는 않습니다.

전달한 내용은 외부 AI 서비스로 전송됩니다. 따라서 비밀번호나 인증 정보처럼 민감한 정보를 함께 보내면 안 됩니다. 이 서비스는 연결용 비밀키를 백엔드 환경변수에 두고, 프롬프트에는 공개해도 되는 통계만 넣습니다.

AI 가 자료를 잘못 읽거나 잘못된 지시를 따를 수 있습니다. 참고 자료 속에 숨은 지시에 AI 가 속는 것을 프롬프트 인젝션이라고 합니다. 이 서비스의 조회 도구는 읽기 전용이고 허용된 키워드와 날짜만 받으므로, AI 가 잘못된 지시를 따르더라도 자료를 수정하거나 정해진 범위 밖의 자료를 조회할 수는 없습니다. 그래도 답변의 숫자와 기준일은 화면의 요약과 비교해 확인해야 합니다.

5 미션 요구사항과 검증 결과

요구사항	구현과 확인 결과
데이터 100 건 이상과 DB 연동	Firestore 운영 데이터 558 건을 확인했습니다.
데이터 관리와 요약	테스트 데이터 한 건의 추가·조회·수정·삭제와 요약 기능을 확인했습니다.
데이터 기반 AI 채팅	데이터 요약 전달과 실제 AI 응답, 답변을 기다리는 동안의 로딩 표시(화면 6)를 확인했습니다.
대화 저장과 불러오기	사용자·AI 메시지 2 건 저장, 새로고침 후 같은 대화 복원, 테스트 대화 삭제를 확인했습니다.
배포와 문서화	Vercel 프런트엔드 커밋 f35cd3f Ready·Production, Render 백엔드 커밋 7bc1366 Live, 배포 후 AI 답변을 확인했습니다. 실행·환경변수·기술 설명과 원본 로그 경로를 문서화했습니다.
보너스 1 도구 연결	Function Calling 1 회 5 건 반환과 외부 MCP 조회 13 건 반환을 확인했습니다.
보너스 2 화면과 자료 활용	평균·최솟값·최댓값·최근/이전 4 개 평균 차이와 추세·순위, 그래프·JSON·다크 모드를 구현했습니다.

필수 제출 화면은 데이터 요약 화면 1, 질문과 답변 화면 4·5·16, 채팅 로딩 화면 6, 대화 불러오기 화면 7, 데이터 관리 화면 8~11 에 정리했습니다. 각 화면 아래에는 확인 내용을 주석으로 달았습니다.

원본 자료를 유지하면서 CRUD 를 검증했습니다

2026 년 10 월 12 일의 퍼스널브랜딩 테스트 자료를 관심도 25 로 추가했습니다. 목록과 새로고침 후 조회에서 확인하고 30 으로 수정한 뒤, 해당 테스트 자료를 삭제했습니다. 원본 558 건은 유지했으며 삭제 후 총 558 건으로 돌아온 것을 확인했습니다. 이 값은 기능 확인을 위한 시험 자료이며 실제 관측값이 아닙니다.

자동화 테스트와 실제 사용 검증을 구분했습니다

자동화 테스트는 계산과 처리 흐름을 검사하고, 외부 AI 와 DB 는 주로 테스트용 가짜 응답(모의 응답)으로 대신합니다. 그래서 실제 AI 응답·운영 DB·배포 화면은 별도로 확인했습니다. 테스트 수와 실행 결과는 5.15 에 정리했습니다.

대화 흐름 테스트는 목록 조회·선택·상세 조회·메시지 복원 및 오류 처리를 검사합니다.

5.1 실행 화면 데이터 요약과 화면 전환

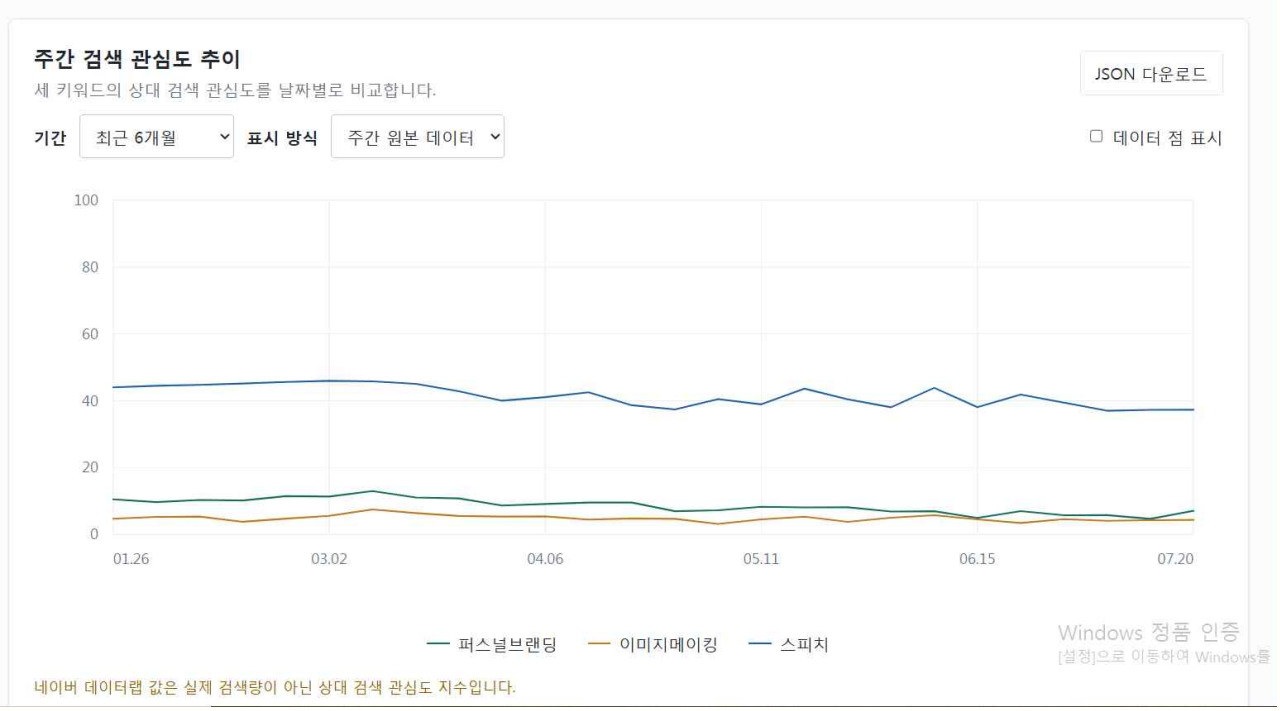


화면 1 데이터 요약입니다. 저장된 기간과 전체 558 건을 확인할 수 있습니다. 키워드별 최신 자료 4 개의 평균은 퍼스널브랜딩 5.85, 이미지메이킹 4.35, 스피치 37.79 입니다. 검색 횟수가 아니라 상대 관심도입니다.



화면 2 같은 자료를 다크 모드로 표시한 모습입니다. 배경과 글자 색이 바뀌어도 기간·자료 수·평균값은 동일합니다. 두 화면은 요약 영역을 촬영한 것으로 아래쪽 일부는 포함되지 않았습니다.

5.2 실행 화면 기간별 그래프



화면 3 최근 6 개월과 주간 원본 데이터를 선택한 수정 후 그래프입니다. 가로축은 날짜, 세로축은 상대 검색 관심도이며 세 키워드가 색으로 구분됩니다. 점 표시는 끈 상태이고 선 아래에 색이 채워지지 않아 기간별 흐름을 선으로 확인할 수 있습니다. 여기서 최근은 오늘이 아니라 저장 자료의 마지막 관측일을 기준으로 합니다.

기간과 표시 방식을 선택해 흐름을 비교합니다. 이 화면은 원본 값이며, 4 주 이동평균을 선택하면 주별 흔들림을 줄인 흐름을 볼 수 있습니다.

5.3 실행 화면 AI 질문과 대화 기록



화면 4 오른쪽에는 질문과 AI 답변, 왼쪽에는 저장된 대화 목록과 메시지 수가 표시됩니다. 세 키워드의 관심도를 비교하고 블로그 방향을 추천해 달라고 질문했습니다. 답변의 평균값 5.85·4.35·37.79 는 요약 화면과 일치합니다.

해석할 때 주의할 점

화면의 "검색량"은 "상대 검색 관심도"로 해석해야 합니다. 평균 기간으로 적힌 6 월 22 일부터 7 월 20 일까지에는 관측일이 5 개 있지만 실제 최신 4 개 관측일은 6 월 29 일·7 월 6 일·7 월 13 일·7 월 20 일입니다. 또 "모두 하락"은 평균값의 수치 감소를 뜻하며, 프로그램의 추세 분류와 구분해야 합니다. 이미지메이킹은 평균이 약 0.36 포인트 낮아졌지만 변화 폭이 0.5 포인트 이하여서 "유지"로 분류됩니다.

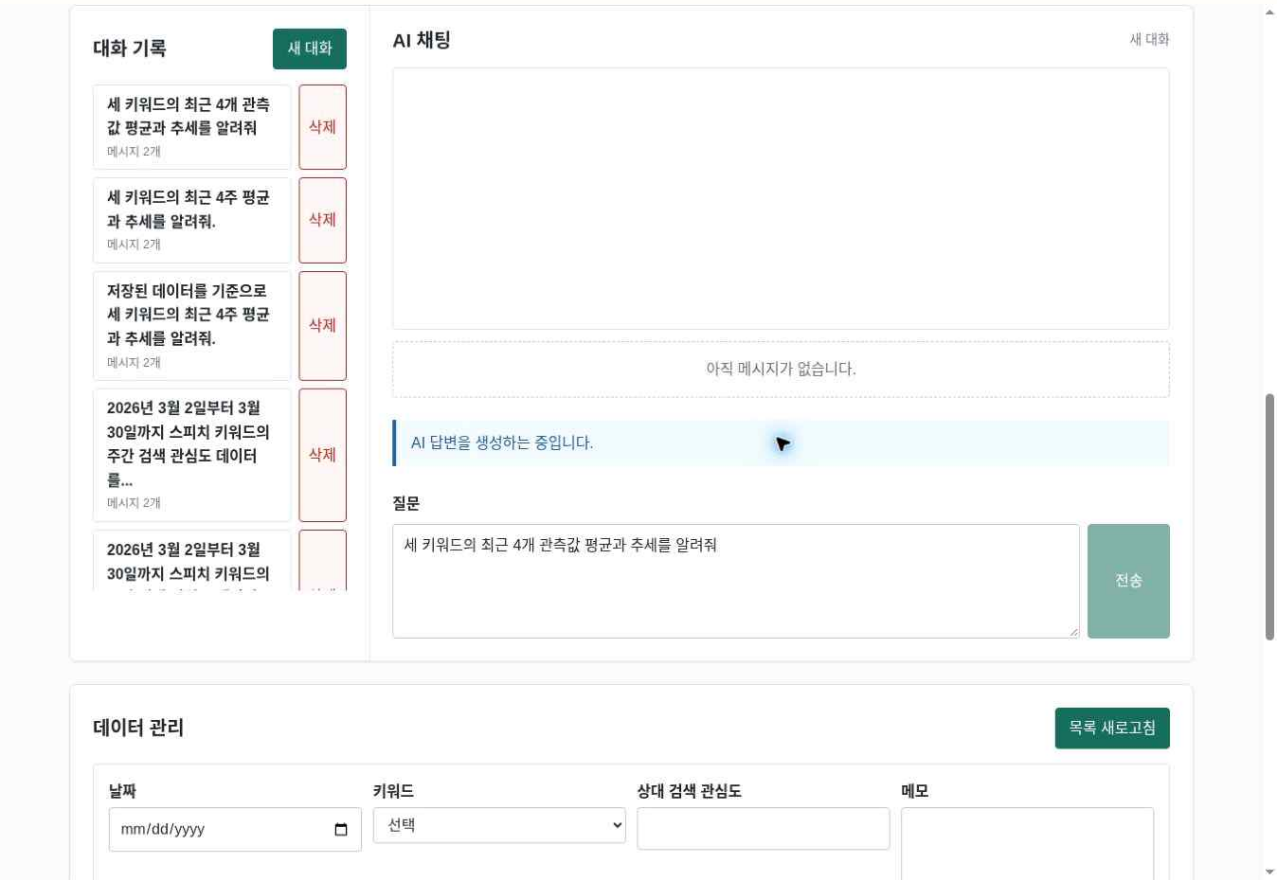
대화 저장과 새로고침 후 복원을 실제 서비스에서 확인했습니다. 대화 선택 후 메시지 복원 화면은 화면 7 에 별도로 수록했습니다.

5.4 실행 화면 데이터 요약과 AI 답변 통합



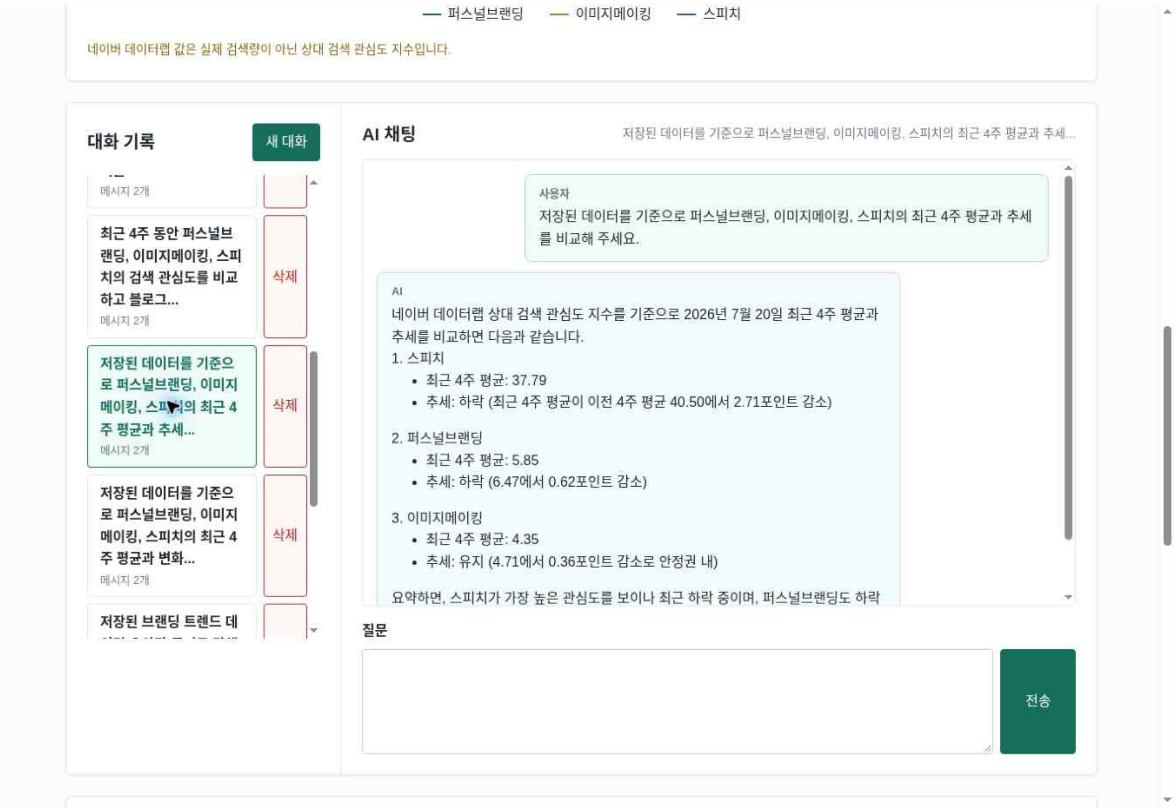
화면 5 데이터 요약, 수정된 선 그래프, 저장된 대화 목록, 질문과 AI 답변이 한 페이지에 함께 표시되는 전체 구성입니다. 과제에서 요구한 '데이터 요약이 보이는 채팅 화면' 증빙이며, 요약 수치는 화면 1, 같은 수치를 사용한 AI 답변은 화면 7 과 16 에서 확대해 확인할 수 있습니다.

5.5 실행 화면 AI 답변 로딩 표시



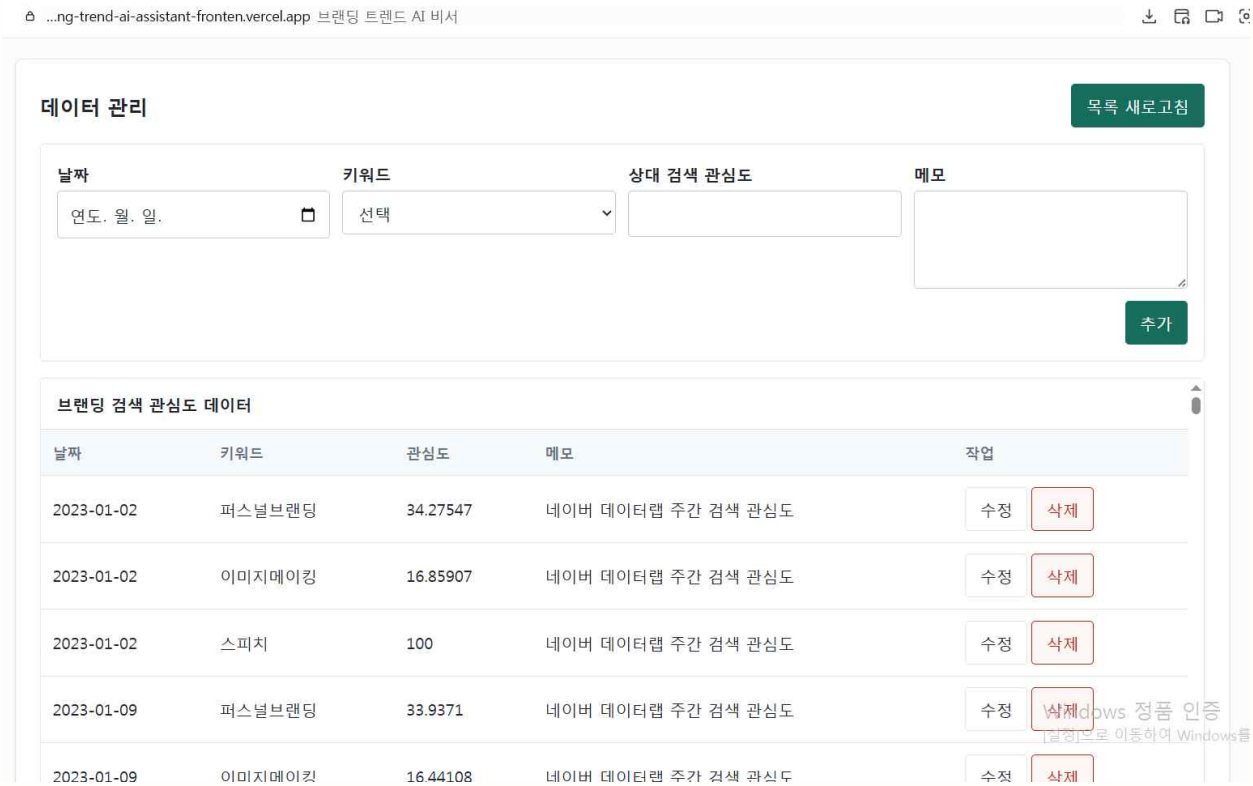
화면 6 질문을 전송한 직후의 채팅 화면입니다. 채팅 영역에 'AI 답변을 생성하는 중입니다.'가 표시되고 전송 버튼은 흐리게 바뀌어 누를 수 없는 상태입니다. 답변을 기다리는 동안 처리 중임을 알 수 있도록 한 로딩 표시입니다.

5.6 실행 화면 저장된 대화 불러오기



화면 7 왼쪽의 초록색 대화를 선택하자 오른쪽에 저장된 질문과 AI 답변이 다시 표시됐습니다. 새 질문을 전송한 화면이 아니라 기존 대화를 선택해 복원한 결과입니다. 요약과 동일하게 스피치 37.79·퍼스널브랜딩 5.85·이미지메이킹 4.35 를 확인할 수 있습니다.

5.7 실행 화면 저장 자료 조회



화면 8 DB 에 저장된 자료를 목록으로 불러온 화면입니다. 한 행에는 날짜·키워드·관심도·메모가 있습니다. 위쪽 입력란으로 자료를 추가하고, 각 행의 버튼으로 수정하거나 삭제할 수 있습니다. 스피치의 100 은 100 번 검색했다는 뜻이 아닙니다.

5.8 실행 화면 테스트 자료 추가



화면 9 목록 맨 아래에 2026 년 10 월 12 일 퍼스널브랜딩 자료를 관심도 25 로 추가했습니다. 메모의 MISSION2_CRUD_TEST 는 실제 관측 자료와 구분하기 위한 표시입니다.

5.9 실행 화면 테스트 자료 수정



화면 10 같은 날짜와 테스트 메모를 가진 자료의 관심도를 25 에서 30 으로 수정했습니다. 새 자료를 또 만든 것이 아니라 기존 한 건의 값이 바뀌었는지 확인한 것입니다.

5.10 실행 화면 테스트 자료 삭제



화면 11 추가했던 2026 년 10 월 12 일 테스트 행이 삭제되고 원래 자료의 마지막 날짜인 2026 년 7 월 20 일까지만 남은 화면입니다. 추가·조회·수정·삭제를 순서대로 확인했고, 삭제 후 전체 자료가 558 건으로 돌아온 것은 별도 요약 검증으로 확인했습니다.

25 와 30 은 기능을 시험하기 위해 넣은 값입니다. 실제 검색 관심도 분석에는 사용하지 않았습니다.

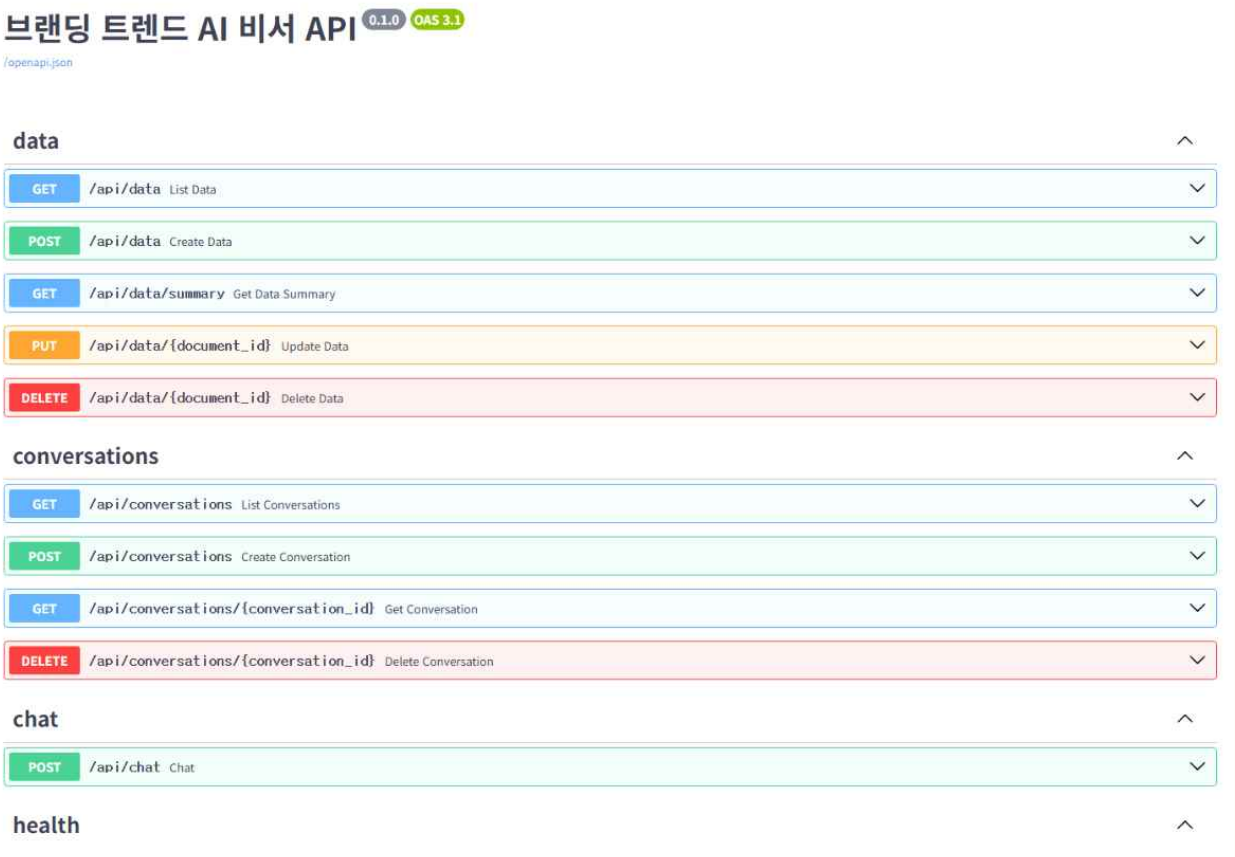
5.11 실행 화면 JSON 파일 다운로드



화면 12 브라우저 다운로드 기록에서 branding-trend-data-2026-10-15.json 파일이 내려받아진 것을 확인했습니다. JSON 은 날짜·키워드·값을 프로그램이 다시 읽기 좋은 형태로 저장하는 파일 형식입니다. 파일명 날짜는 내보내기 때 붙은 이름이며 데이터의 마지막 관측일과는 다릅니다.

그래프 화면의 JSON 다운로드 버튼으로 저장된 자료를 파일로 가져가 다른 분석에 사용할 수 있습니다.

5.12 실행 화면 API 설명서



화면 13 Swagger API 설명서입니다. data 는 자료 관리, conversations 는 대화 기록, chat 은 AI 질문을 처리하는 요청입니다. GET 은 조회, POST 는 생성이나 요청 전송, PUT 은 수정, DELETE 는 삭제에 해당합니다. 화면 뒤에서 어떤 일을 요청할 수 있는지 목록으로 확인하는 곳입니다.

각 요청은 자료 관리, 대화 관리, AI 질문 처리로 나뉩니다. 기능별 실행 결과는 앞의 화면과 검증 기록에 정리했습니다.

5.13 검증 기록 Function Calling 실행 로그

저장소 최종보고서에 수록된 실행 로그 원문 발췌입니다.

```
function_calling_response tool_call_count=1 tool_names=get_weekly_interest_data
trend_tool_success tool_name=get_weekly_interest_data returned_count=5
```

첫 줄의 tool_call_count=1 은 AI 가 조회 기능을 한 번 요청했다는 뜻입니다. tool_names 에는 사용한 도구 이름이 표시됩니다. 둘째 줄의 trend_tool_success 는 조회 성공을, returned_count=5 는 자료 다섯 건을 돌려줬다는 뜻입니다.

출처 GitHub MISSION2_FINAL_REPORT.md 의 Function Calling 실행 기록
기록 보존 커밋 c9afe25 코드 보완 커밋 7bc1366

https://github.com/coolgirl2046/branding-trend-ai-assistant/blob/c9afe25bf3f4968f39a8c3771bb4b7a4116510e6/MISSION2_FINAL_REPORT.md

5.14 검증 기록 외부 MCP 조회

Claude Code 가 남긴 실행 기록 원본(stream-json 형식)과, 한글이 깨지지 않도록 변환한 파일(UTF-8)을 커밋 7bc1366 에 보관하고 두 파일을 대조했습니다. 15 개 기록(이벤트)의 내용과 순서가 같았습니다. 아래는 실제 조회 결과에서 필요한 항목만 뽑은 것입니다.

도구 mcp__branding-trend-ai-assistant__query_weekly_interest
입력 keyword=퍼스널브랜딩
start_date=2026-01-01, end_date=2026-03-31
반환 "observation_count":13
첫 자료 {"date":"2026-01-05","keyword":"퍼스널브랜딩","value":11.46496}
끝 자료 {"date":"2026-03-30","keyword":"퍼스널브랜딩","value":8.67834}
최종 이벤트 "type":"result", "subtype":"success", "is_error":false

주간 자료 조회 도구를 1 번 호출했고, 같은 호출 번호(식별자)로 결과가 1 번 돌아왔습니다. 도구를 찾는 과정(ToolSearch)은 자료 조회가 아니므로 횟수에 넣지 않았습니다. 돌려받은 자료는 13 건이며 위에는 첫 자료와 끝 자료만 옮겼습니다. 연결 기록에서는 프로젝트 MCP 서버가 연결됨(Connected) 상태인 것도 확인했습니다.

이 증빙은 외부 AI 프로그램이 실제로 조회한 원본 기록입니다. 그림으로 다시 만들지 않고 조회 조건·결과·성공 여부를 그대로 옮겼으며, 원본 파일 위치를 함께 적었습니다.

출처 커밋 7bc1366 의 evidence/mcp_connection_oct19.txt, mcp_query_weekly_interest_oct19.jsonl 및 mcp_query_weekly_interest_oct19_utf8.json

https://github.com/coolgirl2046/branding-trend-ai-assistant/tree/7bc1366/evidence

5.15 검증 기록 자동화 테스트

자동화 테스트는 프로그램이 정해진 대로 동작하는지 자동으로 검사하는 코드입니다. 커밋 7bc1366 에 포함된 실행 기록에서 백엔드 85 개와 프런트엔드 23 개, 총 108 개가 통과한 것을 확인했습니다. 외부 AI 와 DB 는 테스트용 가짜 응답(모의 응답)으로 대신해 검사하므로, 실제 운영 확인은 배포 화면에서 따로 했습니다.

대상	검증 결과	검증 범위
백엔드	85 개 통과	입력 검증·데이터 API·요약·채팅·도구 연결·변환
프런트엔드	23 개 통과	그래프·이동평균·JSON·다크 모드·대화 흐름
합계	108 개 통과	보완 코드 로컬 실행 기준

출처 evidence/backend_tests_oct19.txt 및 frontend_tests_oct19.txt

검증 기준 커밋 7bc1366 원본 로그 파일명은 oct19 로 보존

테스트 실행과 실제 출력

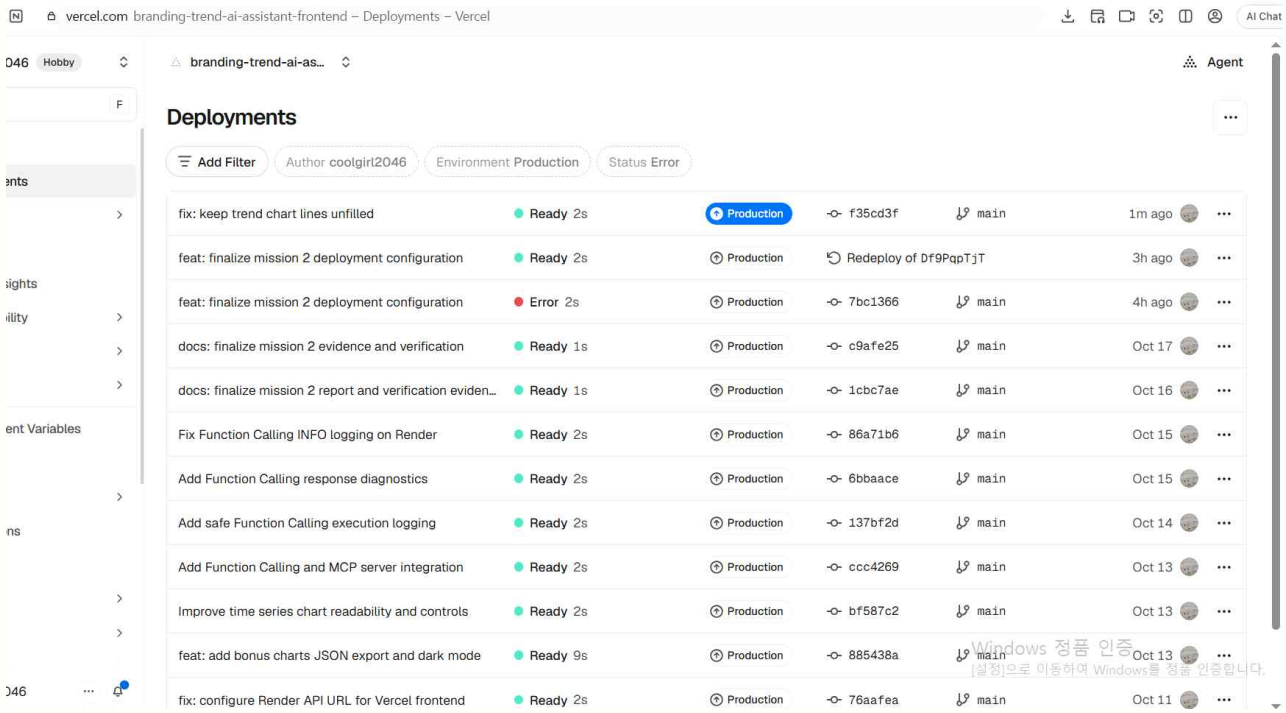
```
백엔드
python -m unittest discover -s backend/tests -v
Ran 85 tests in 12.926s
OK

프런트엔드
node --test frontend/tests/*.test.mjs
tests 23
pass 23
fail 0
skipped 0
```

테스트에서는 답변 길이 제한 설정이 올바르게 읽히고 실제 AI 요청에 전달되는지, 도구 사용 뒤 이어지는 요청에도 같은 제한이 적용되는지, Vercel 배포 때 API 주소 파일이 만들어지고 잘못된 주소는 거절되는지를 확인했습니다.

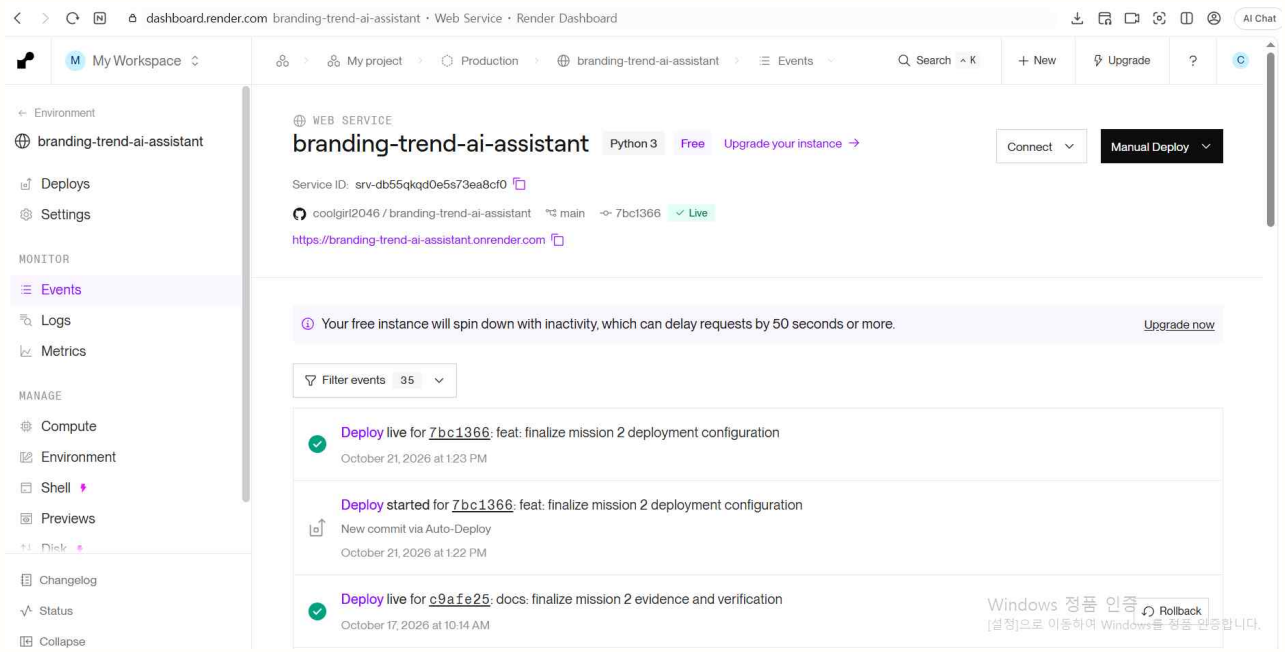
5.16 실행 화면 Vercel 최종 배포

화면 14~16 은 2026 년 10 월 21 일 촬영한 실제 배포·서비스 화면입니다.



화면 14 는 그래프 선 채우기 오류를 수정한 프런트엔드 커밋 f35cd3f 의 배포가 Ready 와 Production 으로 표시된 화면입니다. 이후 커밋에서는 frontend/와 backend/ 앱 코드는 변경하지 않고, README·제출 증빙·보고서 PDF 만 추가·수정했습니다. 세 번째 줄의 Error 는 API_BASE_URL 이 없었던 7bc1366 첫 빌드 실패이며, 환경변수 설정 후 성공한 과정은 6 장에서 설명했습니다.

5.17 실행 화면 Render 최신 배포



화면 15 Render 상단에 백엔드 보완 커밋 7bc1366 과 Live 가 함께 표시됩니다. Events 의 Deploy live for 7bc1366 기록으로 백엔드 배포를 확인했습니다. 이후의 f35cd3f 는 프런트엔드 CSS 한 파일만 수정한 커밋입니다.

5.18 실행 화면 재배포 후 AI 답변



화면 16 최신 프런트엔드 배포 후 저장된 대화를 선택한 상태에서 세 키워드의 최근 4 개 관측값 평균과 추세를 질문한 결과입니다. 답변의 원수치는 스피치 37.78861·퍼스널브랜딩 5.84693·이미지메이킹 4.35410 이며, 요약 화면에서는 각각 37.79·5.85·4.35 로 표시됩니다. 자료 기준일은 2026 년 7 월 20 일입니다.

6 문제 해결과 배운 점

발생한 문제	해결 과정과 확인한 점
AI 조회 기능의 실행 로그가 보이지 않았습니다	서버의 기록 설정 때문에 일반 실행 기록이 표시되지 않았습니다. 설정을 수정하고 관련 테스트를 추가해 도구 호출 여부를 확인할 수 있게 했습니다.
로그에 조회 조건이 남을 가능성이 있었습니다	질문이나 도구 인자 대신 도구 이름과 반환 건수만 기록하도록 수정하고 테스트했습니다.
채팅 자동 검증 중 요청이 차단되었습니다	자동 검증 과정에서 테스트 대화를 먼저 선택하지 않아, 어떤 대화인지 구분하는 정보가 빠졌습니다. 대화를 선택하고 상세 정보를 읽은 뒤 질문하도록 고쳐 정상 응답을 확인했습니다.
Vercel 최신 배포가 실패했습니다	배포 기록(Build Logs)에서 API_BASE_URL 누락을 확인했습니다. Production 환경변수에 Render API 주소를 저장하고 재배포해 Ready·Production 전환과 실제 AI 답변을 확인했습니다.
그래프 선 아래가 색으로 채워졌습니다	선과 점에 같은 색상 규칙이 적용되면서, 선 안쪽을 칠하지 말라는 설정(fill: none)이 덮어써진 문제였습니다. 규칙이 선에만 적용되도록 범위를 좁혀(polyline.trend-chart-line) 수정했고, 프런트엔드 테스트 23 개 통과와 Vercel 커밋 f35cd3f 배포 후 선 그래프 화면으로 다시 확인했습니다.

AI 활용 과정에서 배운 점

이번에는 AI 에게 코드 생성을 요청하는 데서 끝내지 않고, 데이터가 저장되고 화면에 표시되며 질문에 활용되는 전체 흐름을 확인했습니다. 특히 "코드가 있다"는 것과 "실제 서비스에서 작동한다"는 것은 구분해야 한다는 점을 배웠습니다.

문제가 생기면 오류 메시지와 실행 결과를 바탕으로 수정하고, 다시 확인하는 방식으로 진행했습니다. 비전공자인 저는 모든 코드를 외우기보다 각 부분의 역할과 데이터가 이동하는 순서를 이해하는 데 중점을 두었습니다.

현재 한계와 다음 개선 방향

자료의 마지막 관측일은 2026 년 7 월 20 일이므로 이후의 실제 트렌드는 이 자료만으로 설명할 수 없습니다. 향후에는 새 자료를 같은 기준으로 추가하고 값이 올바른지 확인하는 절차를 보완할 수 있습니다.

상대 관심도만으로 변화의 원인을 확정할 수 없으며, AI 해석도 수치와 비교해 확인해야 합니다. 실제 답변에서도 검색량이라는 표현과 평균 기간 설명에 부정확한 부분이 있어 해당 화면 아래에 주석으로 구분했습니다. 향후에는 기간과 용어가 정확한지 확인하는 응답 검증을 보완할 필요가 있습니다.

이번 미션을 통해 분석 자료를 저장하고, 화면에서 확인하고, AI 질문에 활용하는 하나의 흐름을

완성했습니다. 저는 기능을 정하고 AI 의 도움으로 구현한 뒤 실행 결과를 확인했습니다. 앞으로도 코드를 생성하는 데서 멈추지 않고, 사용 목적에 맞게 작동하는지 검증하는 방식으로 결과물을 만들고자 합니다.

부록 예상 질문과 답변

무엇을 만든 서비스인가요

브랜딩 관련 검색 관심도 자료를 그래프와 AI 질문으로 확인하는 서비스입니다. 미션 1 에서 분석한 자료를 저장해 두고, 사용자가 직접 조회하고 질문할 수 있도록 만들었습니다.

일반 ChatGPT 와 어떤 차이가 있나요

제 서비스는 질문과 함께 저장된 자료의 요약을 AI 에 전달합니다. 특정 기간의 원자료가 필요하면 조회 도구로 가져와 답변에 참고하도록 연결했습니다.

AI 를 따로 학습시킨 건가요

이 프로젝트에서는 제 자료로 AI 모델을 새로 학습시킨 것이 아닙니다. 질문할 때 참고 자료를 전달하고, 필요하면 저장소를 조회하게 했습니다.

DB 와 API 는 무엇인가요

DB 는 자료를 보관하는 저장소입니다. API 는 화면과 처리 프로그램, AI 등이 서로 요청과 결과를 주고받는 연결 방식입니다.

자료가 정확하게 관리되는지 어떻게 확인했나요

원본과 별도로 테스트 자료를 한 건 넣어 조회하고, 값을 바꾼 뒤 삭제했습니다. 마지막에 원본 데이터가 558 건으로 유지되는지 확인했습니다.

최근 4 주 평균이라고 해도 되나요

최신 자료 네 개의 평균입니다. 자료가 빠짐없이 매주 기록되어 있을 때 최근 4 주 평균이 됩니다. 오늘을 기준으로 한 최근 4 주는 아닙니다.

본인이 코딩한 부분은 무엇인가요

코드 구현과 수정에는 Codex 의 도움을 받았습니다. 저는 기능과 확인할 내용을 정하고, 실행 결과를 보면서 수정과 검증을 진행했습니다.

완성됐다는 근거는 무엇인가요

자동화 테스트의 통과 기록과 실제 배포 화면 검증이 있습니다. 채팅 저장·불러오기, 데이터 추가·조회·수정·삭제, AI 의 자료 조회 기능을 확인했습니다.

컨텍스트 주입의 장점과 단점은 무엇인가요

제 자료의 요약을 AI 에 함께 전달해 상황에 맞는 답변을 받을 수 있다는 점이 장점입니다. 다만 요약에 없는 세부 내용은 별도 조회가 필요하고, 민감한 정보는 보내면 안 되며, AI 도 틀릴 수 있으므로 숫자를 직접 확인해야 합니다.

부록 도구 연결과 활용

Function Calling 과 MCP 는 어떻게 다른가요

Function Calling 은 이 서비스 안에서 AI 가 필요한 자료를 조회해 달라고 요청하는 기능입니다. MCP 는 외부 AI 프로그램에서도 같은 조회 기능을 연결해 쓰게 하는 공통 규격입니다. 둘 다 제 프로젝트에서는 읽기 전용으로 자료를 가져오는 데 사용했습니다.

이 서비스를 어디에 활용할 수 있나요

브랜딩 관련 콘텐츠를 기획할 때 저장된 검색 관심도의 흐름을 참고할 수 있습니다. 기간을 바꿔 그래프를 확인하고, 특정 시기의 자료를 AI 에게 질문하는 방식입니다. 현재 유행을 판단하려면 자료를 추가로 갱신해야 합니다.

API 키는 왜 따로 관리하나요

API 키는 외부 서비스를 사용하기 위한 비밀 인증 정보입니다. 화면 코드나 GitHub 에 들어가지 않도록 백엔드의 별도 설정으로 관리했습니다. 비밀번호를 공개된 문서에 적지 않고 따로 보관하는 것과 비슷합니다.